

PRD: Delivery Health & Bottleneck Intelligence Platform

Team size: 2 engineers Timeline: 20 working days Version: 1.0 Date: July 13, 2026

1. Problem Statement

A remote-first engineering company scatters standup notes, sprint velocity, and blocker updates across separate tools (Slack, Jira, standup bots, docs). Because this data is fragmented, leadership has no single lens to tell whether a slowdown is a **temporary coordination hiccup** (someone on PTO, a flaky CI pipeline) or a **systemic delivery bottleneck** (recurring dependency waits, chronic under-resourcing, a broken process). Without that distinction, leadership either over-reacts to noise or under-reacts to real structural problems, and interventions land in the wrong place.

2. Goal

Build a lightweight platform that ingests standup, velocity, and blocker data from multiple sources, classifies issues as **transient vs. systemic**, and gives leadership a clear, trustworthy view of where real bottlenecks live — without adding new reporting overhead for engineers.

3. Success Metrics

- Leadership can correctly identify a systemic bottleneck (validated against a known historical case) using the dashboard alone.
- **80%** of blockers are auto-tagged transient/systemic with an explainable reason.
- Time to detect a recurring blocker pattern drops from "anecdotal / weeks" to "visible within 1 sprint."
- Engineers spend **zero additional minutes/day** on manual reporting (data comes from existing tools).

4. Target Users

- **Primary:** Engineering leadership (VP Eng, Directors, EMs) — need the aggregated, classified view.
- **Secondary:** Team leads / Scrum masters — need to drill into their team's blockers.
- **Not targeted (v1):** Individual contributors as active dashboard users (they're a data source, not a consumer).

5. Scope

In scope (v1 / this 20-day build)

- Ingestion connectors: Jira (velocity + issue/blocker data), Slack (standup channel messages), and a simple standup-bot/CSV import as a fallback.
- Normalization layer: unify "blocker," "impediment," "waiting on X" language into one schema.
- Classification engine: rule-based + simple heuristic scoring to label items **Transient** vs **Systemic**, with a recurrence/duration signal (e.g., same blocker type appearing 3+ sprints = systemic).
- Leadership dashboard: sprint velocity trend, active blockers by type, transient vs systemic breakdown, team-level drill-down.
- Basic alerting: flag when a blocker crosses the systemic threshold.

Out of scope (v1)

- Predictive ML models (v2 candidate).
- Two-way write-back into Jira/Slack.
- Mobile app.
- Custom integrations beyond Jira/Slack (e.g., Linear, Asana) — architected for later, not built now.

6. Functional Requirements

#	Requirement	Priority
F1	Pull sprint velocity + issue data from Jira via API	P0
F2	Pull standup/blocker mentions from a designated Slack channel via API	P0
F3	CSV/manual import fallback for teams not on Jira/Slack	P1
F4	Normalize all blockers into a common schema (team, date, description, category, source)	P0
F5	Classify each blocker as Transient/Systemic based on recurrence, duration, and cross-team pattern	P0
F6	Dashboard: velocity trend per team	P0
F7	Dashboard: blocker breakdown (transient vs systemic) with drill-down by team/category	P0
F8	Alert/flag when a blocker recurs beyond threshold (config: default 3 sprints)	P1

#	Requirement	Priority
F9	Simple auth (leadership login)	P1

7. Non-Functional Requirements

- Data refresh at least once/day (near-real-time not required for v1).
- Read-only integrations only (no write access needed → lower security risk, faster approval).
- Must not require engineers to change their existing workflow.

8. Classification Logic (v1 heuristic, not ML)

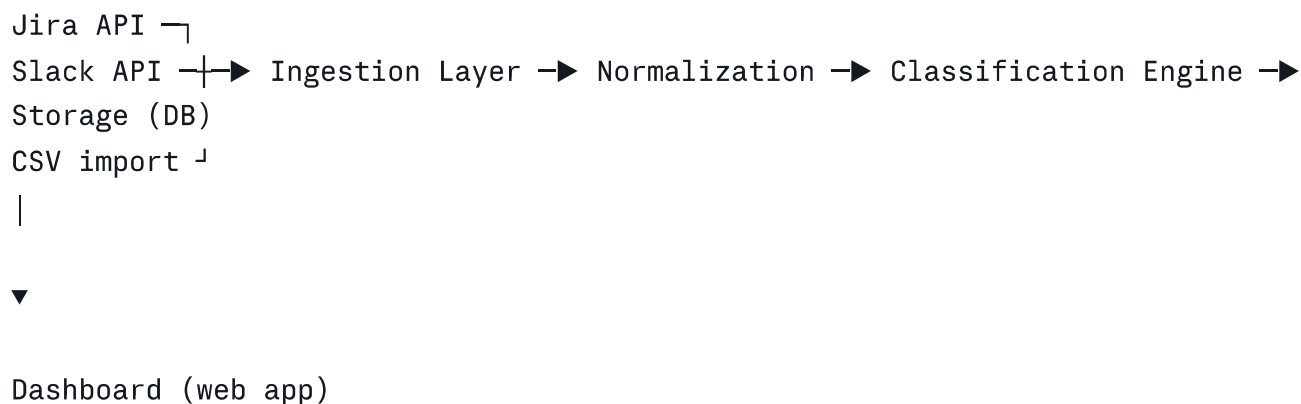
A blocker is scored **Systemic** if it meets ≥ 2 of:

1. Same root-cause category recurs across ≥ 3 sprints for a team.
2. Same category appears across ≥ 2 different teams in the same sprint.
3. Average resolution time for that category exceeds a configurable threshold (e.g., >5 days).
4. It's explicitly tied to a dependency (another team/system) rather than an individual's availability.

Otherwise it's **Transient** (e.g., single mention, resolved in <2 days, tied to PTO/one-off outage).

This keeps v1 explainable — leadership can see *why* something was tagged systemic, which builds trust before you introduce any ML later.

9. High-Level Architecture



10. Suggested Tech Stack (optimized for 2 people / 20 days)

- **Backend:** Python (FastAPI) — fast to write connectors + scoring logic.
 - **DB:** PostgreSQL (or SQLite for the prototype, swap later).
 - **Frontend:** React + a charting lib (Recharts) — single dashboard app.
 - **Scheduler:** simple cron job / APScheduler for daily ingestion.
 - **Auth:** basic email/password or magic link (skip SSO for v1).
 - **Hosting:** Render/Railway/Fly.io for speed — avoid infra overhead with 2 people.
-

11. Team Split (2 people)

Given the two biggest independent tracks are **data (ingestion/classification/backend)** and **experience (dashboard/frontend + integration polish)**, split by track rather than by day, so you're not blocked on each other:

- **Engineer A — Data & Backend:** Jira/Slack connectors, normalization schema, classification engine, DB, API endpoints.
- **Engineer B — Product & Frontend:** Dashboard UI, auth, alerting UI, PRD-to-demo narrative, plus CSV import fallback (F3) since it's UI-adjacent.

Both pair on schema design (Day 1-2) and on final integration/demo (Day 18-20).

12. 20-Day Execution Plan

Phase 1 — Foundation (Days 1-4)

Day	Engineer A (Backend/Data)	Engineer B (Frontend/Product)
1	Finalize data schema (blockers, velocity, teams) together; set up repo, DB	Same — schema workshop; set up Figma/wireframe for dashboard
2	Jira API auth + pull sprint/velocity data (raw)	Wireframe dashboard screens (velocity view, blocker view); set up React skeleton
3	Slack API auth + pull messages from standup channel (raw)	Auth flow (login) scaffolding
4	Normalize Jira + Slack data into common schema	Build API client layer in frontend (mock data first)

Phase 2 — Core Logic & UI (Days 5-10)

Day	Engineer A	Engineer B
5	Build classification engine v1 (recurrence rule)	Velocity trend chart (using mock data)
6	Add cross-team + duration rules to classification	Blocker breakdown chart (transient vs systemic)
7	Persist classified blockers to DB; write API endpoints (GET blockers, GET velocity)	Connect frontend charts to real API endpoints
8	CSV import parser (backend side, for F3)	Team drill-down view UI
9	Scheduler: daily ingestion job (cron)	CSV upload UI (frontend for F3)
10	Mid-point sync/demo: integration test of full pipeline (Jira→classify→DB→API)	Polish dashboard layout, empty/loading states

Phase 3 — Alerting, Auth, Hardening (Days 11-15)

Day	Engineer A	Engineer B
11	Alert threshold logic (config + trigger)	Alert banner/notification UI on dashboard
12	Config UI backend (thresholds editable)	Auth backend integration (real login, sessions)
13	Error handling + logging across connectors	Team/category filter UI on dashboard
14	Rate-limit handling + retry logic for Jira/Slack APIs	"Why is this systemic?" explainability panel (shows the 4 rule signals)
15	Write tests for classification engine (edge cases)	Responsive/UI polish pass

Phase 4 — Integration, Testing, Demo Prep (Days 16-20)

Day	Engineer A	Engineer B
16	End-to-end test with real (or realistic seeded) data across 3+ sprints	End-to-end UI walkthrough with real data
17	Bug fixing from E2E test	Bug fixing from UI walkthrough
18	Deploy backend + DB to hosting	Deploy frontend, connect to live backend
19	Joint: full regression pass, fix deployment issues	Joint: full regression pass
20	Joint: prep demo narrative, walk through the "temporary vs systemic" story with sample data, final PRD/README polish	Same

13. Risks & Assumptions

- **Assumption:** Jira and Slack API access/credentials can be obtained quickly (day 1-2) — if not, this is the biggest schedule risk; have CSV import as fallback demo path.
- **Risk:** Classification rules may need tuning after seeing real data — budget buffer inside Days 15-17 for this.
- **Risk:** With only 2 people, scope creep is the main threat — hold the line on "out of scope" list above until v1 ships.
- **Assumption:** One Slack channel per team is used consistently for standups (if not, ingestion gets messier — worth confirming with pilot teams before Day 3).

14. Definition of Done (v1)

- Leadership can log in, see velocity trend + blocker breakdown for at least 2 real (or seeded) teams across 3+ sprints, and see a clear, explainable transient/systemic tag on each blocker.